

IAT 隐藏验证（静态分析检验）

前几章我们实现了 GetProcAddressR，宣称“二进制里没有明文 API 名”。但宣称不等于事实。本章节的核心任务只有一个：用相关工具打开编译好的 EXE，亲眼验证 IAT（导入地址表）和字符串池里到底还剩下什么。

● IAT 是什么

IAT = Import Address Table（导入地址表）。

- 它是 PE 文件里的一张表格，记录“本程序需要从哪些 DLL 导入哪些函数”。
- 程序启动时，Windows 加载器根据 IAT，把每个函数的真实地址填进去。
- 杀软做静态扫描时，首先看 IAT。

举例：如果你用了 CreateRemoteThread，IAT 里就会出现：

```
KERNEL32.dll
    CreateRemoteThread ← 敏感函数名，直接暴露行为
    VirtualAllocEx
    WriteProcessMemory
```

结论：IAT 里有敏感函数名 = 静态扫描直接拉黑。

● GetProcAddressR 如何隐藏 IAT

GetProcAddressR 不通过系统的导入机制，而是自己手工遍历导出表。这意味着：

- 代码里没有 CreateRemoteThread(...) 这种直接调用。
- 链接器不会把 CreateRemoteThread 填进 IAT。

结果：IAT 里干干净净。

但问题在于：字符串依然可能残留。

● IAT 藏住了，字符串呢

即使 IAT 干净了，字符串依然可能藏在 .rdata 段里。取决于你如何写代码：

(一) 版本 A: Test1.cpp（用 Ror13hash("明文")）

```
DWORD kernel32Hash = Ror13hash("kernel32.dll");
DWORD hashCRT = Ror13hash("CreateRemoteThread");
```

现象：编译器在 /O2 优化下，通常会把字符串常量算成哈希值，然后把字符串丢弃。但如果没开 /O2，编译器可能把字符串保留在 .rdata 里。

验证结果：

- IAT 干净
- .rdata 里还躺着 VirtualAllocEx 和 CreateRemoteThread

原因：因为在 main 里写了：

```
FARPROC checkCRT = GetProcAddress(hKernel32, "CreateRemoteThread");
FARPROC checkVAE = GetProcAddress(hKernel32, "VirtualAllocEx");
```

这两行是“交叉验证代码”，它们直接使用了明文，把字符串写进了二进制。

(二) 版本 B: Test2.cpp（用硬编码哈希值）

```
DWORD kernel32Hash = 2414663231;
DWORD hashCRT = 1929223661;
```

现象：代码里没有任何字符串常量，只有数字。无论开不开 /O2，.rdata 里都不会有 CreateRemoteThread。

验证结果：

- IAT 干净

➤ .rdata 干净 (除了少数不可避免的系统字符串, 比如 "pause")

● 两版代码对比

对比项	Test1.cpp (哈希计算)	Test2.cpp (硬编码哈希)
IAT 敏感函数名	无	无
.rdata 敏感字符串	有 (用了 GetProcAddress(明文) 验证)	无
编译期依赖	需要编译器优化掉字符串	编译器无需优化, 字符串根本不存在
二进制体积	略大	略小
隐蔽性	中	高

结论: Test2.cpp 是更彻底的隐蔽方案。它不在代码里留下任何明文, 一切通过预先算好的哈希值完成。

● 为什么“硬编码哈希值”更彻底

版本 A (Ror13hash("明文")):

- 依赖编译器在 /O2 下把字符串常量算成哈希并丢弃。
- 但编译器优化并不可靠。尤其当字符串被多个函数引用时, 可能保留在二进制里。
- Test1 正是因为 GetProcAddress 的验证代码保留了字符串, 导致字符串泄漏。

版本 B (硬编码哈希值):

- 字符串从不存在, 编译器连“算哈希”这一步都省了。
- 无论是否开 /O2, 二进制里都只有数字。
- 这就是为什么 Test2 的隐蔽性更高。